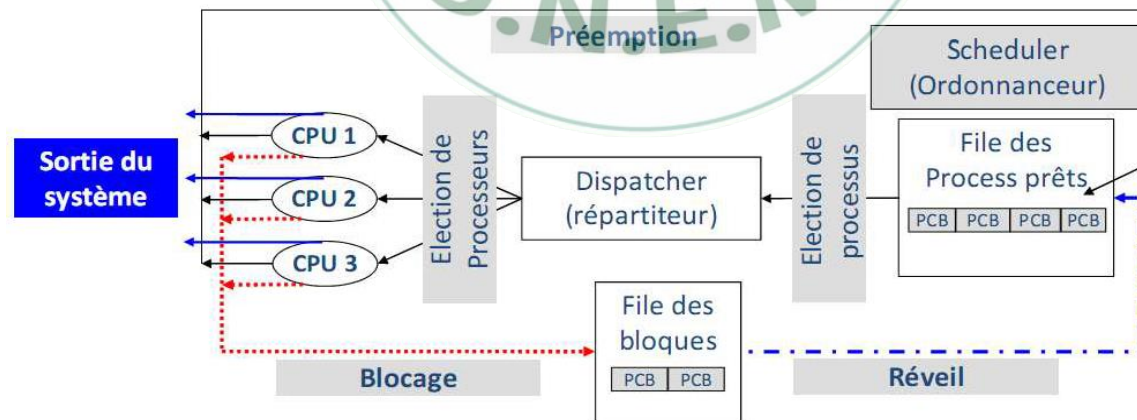


Scheduling/Scheduler

C'est un module (programme) du système d'exploitation qui attribue le contrôle de l'unité centrale, à tour de rôles, aux différents processus en compétition suivant une politique définie à l'avance par les concepteurs du système.



Politiques (Algorithmes) de scheduling



Politiques de schudeling sans préemption

□ **Temps d'attente** : Temps passé à attendre dans la file d'attente des processus prêts.

□ **Temps de sejour** ($T = T_{Fin} - T_{Arrivé}$) où:

✓ T_{Fin} est le temps de fin d'exécution du processus

✓ $T_{Arrivé}$ est son temps d'arrivée

■ **Premier arrive, premier servi (FIFO;FCFS)**

✓ La queue (file d'attente) des processus prêts gérée en FIFO

✓ Exemple (03 processus P_1 , P_2 et P_3):

⌚ Ordre d'arrivée ($t=0$)

⌚ Durées d'exécution sont respectivement : 24, 3, 3 ms

✓ **Diagramme de Gantt**



✓ Temps d'attente: $P_1 = 0$ ms; $P_2 = 24$ ms; $P_3 = 27$ ms

✓ Temps d'attente moyen: $(0 + 24 + 27) / 3 = 17$ ms

Politiques (Algorithmes) de scheduling

Politiques de scheduling sans préemption

► Premier arrive, premier servi (FIFO;FCFS)

✓ Exemple (03 processus P_1 , P_2 et P_3):

⌚ Ordre d'arrivée: P_2 , P_3 , P_1 .

✓ *Diagramme de Gantt*



✓ Temps d'attente: $P_1 = 6$ ms; $P_2 = 0$ ms; $P_3 = 3$ ms

✓ Temps d'attente moyen: $(6 + 0 + 3) / 3 = 3$ ms

✓ *Critique de la méthode*

La politique FIFO désavantage les processus courts s'ils ne sont pas exécutés en premier.

Politiques (Algorithmes) de scheduling

Politiques de scheduling sans préemption

♦ Algorithme du Plus Court d'abord en anglais Short Job First (SJF)

✓ Affecter le processeur au processus possédant le temps d'exécution le plus court

✓ Exemple :

Processus	Temps d'arrivée	Temps d'exécution
P1	0	10
P2	3	4
P3	5	1

✓ *Diagramme de Gantt*



✓ Temps d'attente moyen: $(0 + 8 + 5)/3 = 4,33$ ms

✓ Temps d'attente moyen (FCFS): $(0+7+9) = 5,33$ ms

✓ *Critique de la méthode*

⌚ Avantage les processus les plus courts → **risque de privation**



Exercice :

5 processus A, B, C, D et E. Leurs temps d'exécution et d'arrivée sont donnés par le tableau suivant :

Processus	Temps d'exécution	Temps d'arrivage
A	3	0
B	6	1
C	4	4
D	2	6
E	1	7

- ▶ Avec la stratégie FIFO et SJF :
- ▶ Afficher le diagramme de Gant
- ▶ Calculer le temps de séjour moyen
- ▶ Calculer le temps d'attente moyen

Politiques (Algorithmes) de scheduling

Politiques de scheduling avec préemption

► Le plus court temps restant le premier (SRTF)

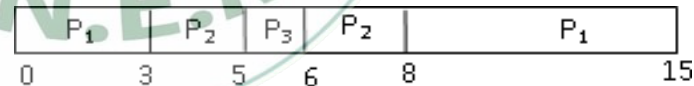
✓ SJF + mécanisme de réquisition.

- ⌚ Mêmes contraintes que SJF.
- ⌚ On choisit le processus qui a le temps restant d'exécution le plus petit.
- ⌚ L'arrivée d'un nouveau processus peut interrompre le processus courant.

✓ Exemple

Processus	Temps d'arrivée	Temps d'exécution
P1	0	10
P2	3	4
P3	5	1

✓ Diagramme de Gantt



✓ Temps d'attente moyen: $(5 + 1 + 0)/3 = 2$ ms

Politiques (Algorithmes) de scheduling

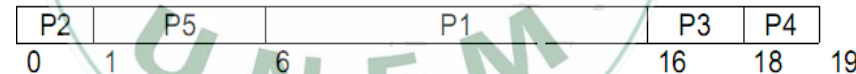
Politiques de scheduling avec préemption

► Scheduling avec priorité

- ✓ Associer à chaque processus une **priorité** (un entier).
- ✓ Affecter le processeur au processus de **plus haute priorité**.
- ✓ Exemple

Processus	Temps d'exécution	Priorité
P1	10	2
P2	1	4
P3	2	2
P4	1	1
P5	5	3

✓ Diagramme de Gantt



✓ Temps d'attente moyen: $(0+1+6+16+18)/5=8.2$ ms

✓ Critique de la méthode

Une situation de blocage peut survenir si les processus de basse priorité attendent indéfiniment le processeur, alors que des processus de haute priorité continuent à affluer.

Politiques (Algorithmes) de scheduling



Politiques de scheduling avec préemption

- Round Robin (Tourniquet ou FIFO Circulaire)
- L'algorithme de scheduling du tourniquet, appelé aussi Round Robin, a été conçu pour des systèmes à temps partagé.
- Il alloue le processeur aux processus à tour de rôle, pendant une tranche de temps appelée quantum.
- Dans la pratique le quantum s'étale entre 10 et 100 ms.
- Exemple : soit les processus suivants : En utilisant un algorithme Round Robin, avec un quantum de 2 ms, on obtient le diagramme de Gantt suivant :

Pi	Instant d'arrivée	Temps d'exécution
A	0	3
B	1	6
C	4	4
D	6	2
E	7	1

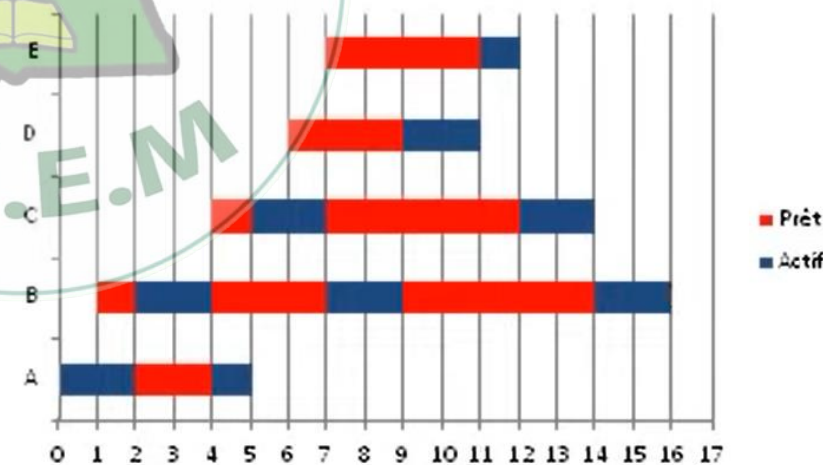
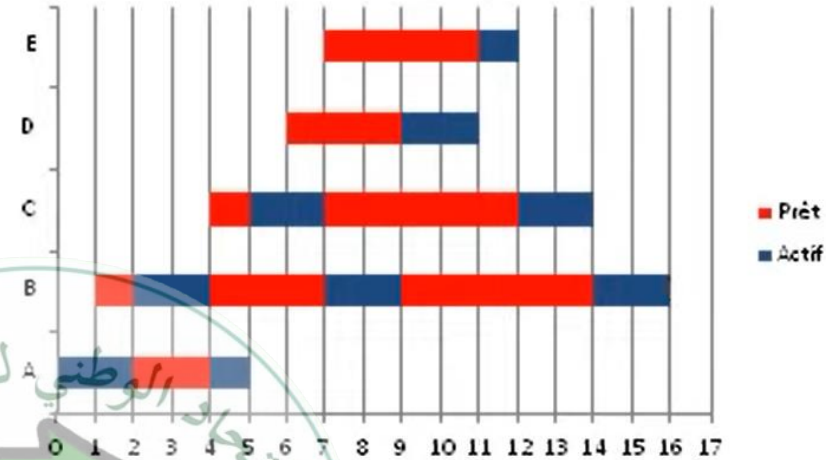


Diagramme de Gantt



	Instant d'arrivée	Temps d'exécution
A	0	3
B	1	6
C	4	4
D	6	2
E	7	1



Temps d'Attente Moyen (Temps passé dans la file d'attente) : $(2+9+6+3+4)/5=4,8$

Temps de Séjour Moyen(avec durée de séjour=T-fin-T-arrivé) : $(5+15+10+5+5)/5=8$

Temps d'accès (Temps de Réponse) : Temps que passe le processus la première fois dans la file d'attente avant d'allouer le processeur

(Début d'exécution-Temps d'arrivée): $0+1+1+3+4=1,8$

Politiques (Algorithmes) de scheduling



Politiques de scheduling avec préemption

- La performance de l'algorithme de Round Robin dépend largement de la taille du quantum. Si le quantum est très grand, la politique Round Robin serait similaire à celle du FCFS. Si le quantum est très petit, la méthode Round Robin permettrait un partage du processeur : Chacun des utilisateurs aurait l'impression de disposer de son propre processeur.





Structure de gestion des processus

Process Control Block (PCB)

- Chaque processus est représenté dans le SE par un PCB (process control block) qui représente le contexte d'un processus

PCB	
Pointeur	État du processus
Numéro du processus	
Compteur d'instructions	
Registres	
Limite de la mémoire	
Liste des fichiers ouverts	
...	

PCB: le Contexte d'un processus est une structure de données qui décrit un processus en cours d'exécution. Ce bloc est créé au même moment que le processus et il est mis à jour en grande partie lors de l'interruption du processus afin de pouvoir reprendre l'exécution du processus ultérieurement.

Table des processus

Table des processus

PID	PCB
1	
2	
...	
n	

PCB du processus n

Compteur ordinal
Registres
État
Priorité
Espace d'adressage
Père
Fils
Fichiers ouverts
Actions associées aux signaux
....

PCB du processus 2

Compteur ordinal
Registres
État
Priorité
Espace d'adressage
Père
Fils
Fichiers ouverts
Actions associées aux signaux
....

PCB du processus 1

Compteur ordinal
Registres
État
Priorité
Espace d'adressage
Père
Fils
Fichiers ouverts
Actions associées aux signaux
....



Structure d'un PCB

Le contexte d'un processus comporte principalement les informations suivantes :

- Le compteur ordinal : adresse de la prochaine instruction à exécuter par le processeur
- Les contenus des registres généraux : ils contiennent les résultats calculés par le processus
- Les registres qui décrivent l'espace qu'il occupe en mémoire centrale (l'adresse de début et de fin par exemple)
- Le registre variable d'état qui indique l'état du processus D'autres informations telles que la valeur de l'horloge, la priorité du processus
- Etc

Communication des processus



Introduction

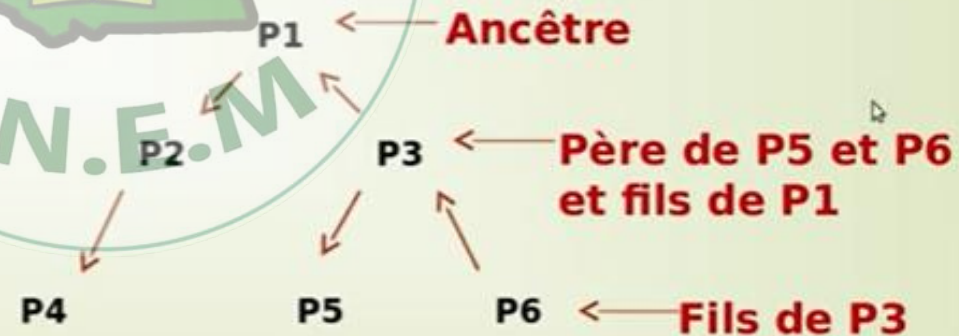
Le système d'exploitation fournit un ensemble d'appels système qui permettent la création, la destruction, la communication et la synchronisation des processus.

Les processus sont créés et détruits dynamiquement.

Un processus peut créer un ou plusieurs processus fils qui, à leur tour, peuvent créer des processus fils sous une forme de structure arborescente.

Le processus créateur est appelé processus **père**.

Le processus créé est appelé processus **fils**.



La hiérarchie des processus





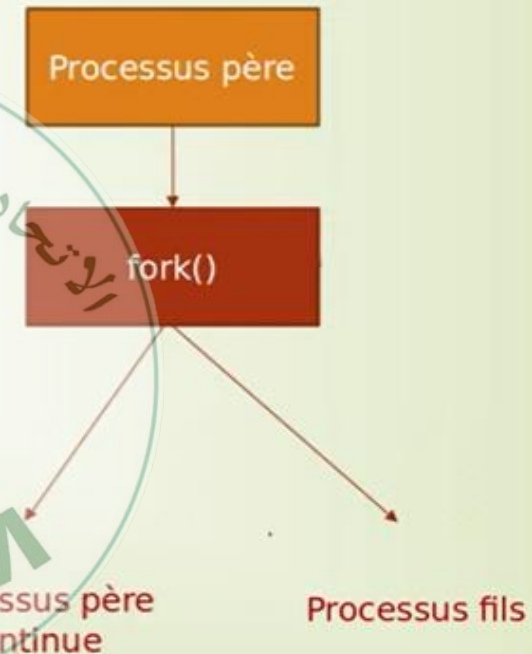
Introduction

Dans certains systèmes (MS-DOS), l'exécution du processus père est suspendue jusqu'à terminaison du processus **fil** → Exécution **séquentielle** → **SE Monotâche**

Dans les systèmes du type Unix ou de Windows, le père continue à s'exécuter en concurrence avec ses fils. → Exécution **asynchrone** → **SE MultiTâche**

Création de processus: fork()

- En langage C sous unix/Linux, la primitive **fork()** sert à créer un nouveau processus (processus fils) en dupliquant le processus appelant (processus père).
- Les deux processus s'exécutent d'une manière concurrente.
- Le processus fils est la copie exacte du processus père, ils **partagent** le même code, le segment de données (variables).



Création de processus: fork()

□ Lors de l'exécution de l'appel-système fork, le noyau effectue les opérations suivantes :

1. Il alloue un bloc de contrôle dans la table des processus.
2. Il copie les informations contenues dans le bloc de contrôle du père dans celui du fils sauf identificateurs (PID, PPID...).
3. Il alloue un PID au processus fils.
4. Il associe au processus fils un segment de code dans son espace d'adressage.
 - Le segment de données et la pile sont également partagés et mis en accès lecture seule.
 - Lors d'un accès en écriture par l'un des deux processus, le segment de données ou la pile sont effectivement dupliqués.
 - Cette technique, nommée « **copie on write** » ou « **copie sur écriture** », permet de réduire le temps de création du processus.
5. Il retourne le pid du processus créé au père et 0 au processus fils.





Création de processus: fork()

Après la création des processus père et fils reprend son exécution après le fork().

- Le code et les données étant strictement identiques, il est nécessaire de disposer d'un mécanisme pour **différencier** le comportement des deux processus après le fork()
- Le code retour du **fork()** est le seul moyen de différencier le père du fils.

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t fork(void);
```

- Les valeurs retournées par la primitive **fork()** sont:
 - PID du fils** dans le processus père
 - 0** dans le processus fils
 - 1** si le fork échoue (par manque de mémoire ou si l'utilisateur a déjà créé trop de processus)





Identification des processus

La primitive **getpid()** renvoie l'identifiant du processus appelant (PID).
La primitive **getppid()** renvoie l'identifiant du processus père du processus appelant (PID du père)

PID : process identifier

PPID : process Parent Identifier



Création de processus: fork()

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Coucou %d \n« , getpid());
    fork(); /* création du processus fils */
    printf(« Mon PID %d: PID père
    %d\n", getpid(), getppid());
}
```



Schéma des processus

Processus
initial
PID=7011

fork()

Processus père
continue
PID=7011

Processus fils
créé
PID=7012

Résultats

Coucou 7011

Mon PID 7012: PID père 7011

Mon PID 7011: PID père 5668

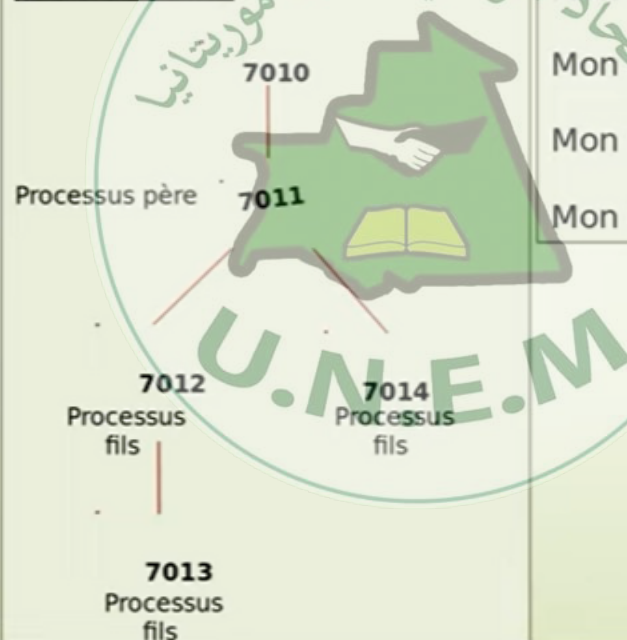


Création de processus: fork()

Exemple2.c

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    printf("Coucou %d \n« , getpid());
    fork(); /* création du processus fils */
    fork(); /* création du processus fils */
    printf(« Mon PID %d: PID père
%d\n", getpid(), getppid());
}
```

Arbre binaire des processus



Résultats

Coucou 7011

Mon PID 7011: PID père 7010

Mon PID 7012: PID père 7011

Mon PID 7014: PID père 7011

Mon PID 7013: PID père 7012

7010 donne naissance a un processus 7011 qui représente le programme



Création de processus: fork()

Exemple3.c (avec la structure if)

Processus père

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main(){
    pid_t pid;
    pid = fork();
    if (pid == -1)
        printf("Echec de création\n");
    else if (pid == 0)
        printf("Hello Dad\n");
    else
        printf("Bonjour fils\n");
}
```

Processus fils

```
if (pid == -1)
    printf("Echec de création\n");
else if (pid == 0)
    printf("Hello Dad\n");
else
    printf("Bonjour fils\n");
```

Résultats

Bonjour fils

Hello Dad





Mort naturelle et Zombie

- Un processus peut se terminer normalement ou anormalement.
- Dans le premier cas, l'application est abandonnée à la demande de l'utilisateur, ou la tâche à accomplir est finie.
- Dans le second cas, **un dysfonctionnement est découvert**, qui est si sérieux qu'il ne permet pas au programme de continuer son travail



Orphelin et Zombie

- **Processus orphelins**

- si un processus père meurt avant son fils ce dernier devient orphelin.

- **Processus zombie**

- Si un fils se termine tout en disposant toujours d'un PID celui-ci devient un processus zombie
- Le cas le plus fréquent : le processus s'est terminé mais son père n'a pas (encore) lu son code de retour.



Exercice

On dispose des 4 processus suivants :

Processus	Temps d'arrivée (ms)	Durée d'exécution (ms)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

L'ordonnanceur applique la politique **FIFO circulaire (Round-Robin)** avec un **quantum de 3 ms**.

Afficher le diagramme de gant
Calculez le temps d'attente moyen
Calculez le temps de séjour moyen

Chapitre IV : Gestion de la mémoire centrale



□ Généralités

□ Objectifs

□ Stratégies d'allocation de la memoir :

- 🕒 Une seule zone contiguë (Mono-programmation)
- 🕒 Système multiprogrammé (Partition multiples)
 - ✓ Compactage
 - ✓ État de la mémoire
 - ✓ Politiques de placement



Généralités



➤ Instruction ou donnée de l'espace d'adressage doit être chargée en mémoire centrale (principale, physique) avant d'être traitée par un processeur.

➤ Le gestionnaire de la mémoire est le composant du système d'exploitation qui se charge de gérer l'allocation d'espace mémoire aux processus à exécuter :

- ⌚ Comment organiser la mémoire ? (une ou plusieurs partitions, ...).
- ⌚ Faut-il allouer une zone contiguë à chaque processus à charger en mémoire?
Faut-il allouer tout l'espace nécessaire à l'exécution du processus entier ? (politique d'allocation).
- ⌚ Comment mémoriser l'état de la mémoire? Parmi les parties libres en mémoire, lesquelles allouées au processus? (politique de placement)
- ⌚ Les adresses figurant dans les instructions sont-elles relatives? Si oui, comment les convertir en adresses physiques.



Objectifs de la mémoire centrale

❑ **Réallocation/Mapping**

- ▶ Chaque processus voit son propre espace mémoire, numéroté à partir de l'adresse 0
cet espace est physiquement implanté à partir d'une adresse quelconque
la possibilité de déplacer un processus en mémoire (lui changer de place)

❑ Protection

- ▶ Un processus ne doit pas modifier les informations d'un autre

❑ Partage

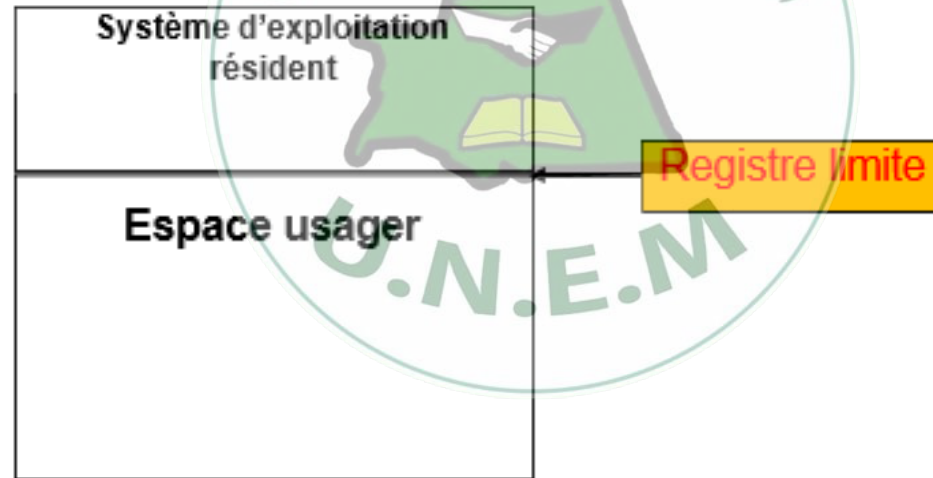
- ▶ Partage d'un espace mémoire (programmes ou des données) entre plusieurs processus.
- ▶ Exemple : un éditeur de texte partagé entre plusieurs utilisateurs d'un système à temps partagé



Stratégies d'allocation de la mémoire

Une seule zone contiguë (**Mono-programmation**)

- Un seul programme à la fois
 - ➔ allocation directe dans l'espace d'adressage physique



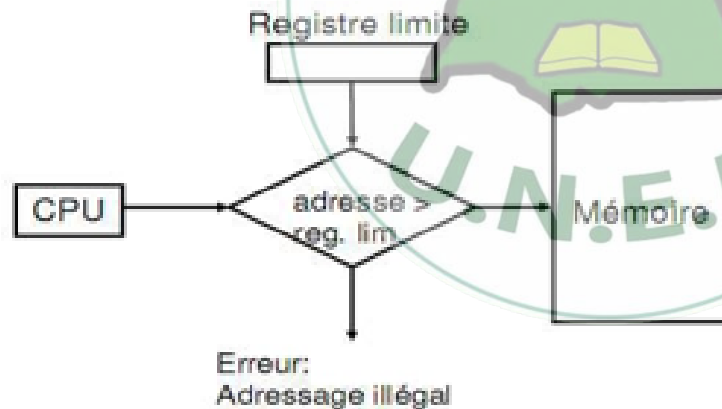
Stratégies d'allocation de la mémoire



Une seule zone contiguë (**Mono-programmation**)

□ Protection

- Pour protéger le code et les données du SE des programmes des utilisateurs, on utilise un **registre limite** qui indique la limite de la zone réservée aux utilisateurs.
- Chaque adresse référencée par un processus est comparée avec le registre limite.



Stratégies d'allocation de la mémoire



Système multiprogrammé

- La mémoire est partagée entre le système d'exploitation et **plusieurs processus**.
- Optimisation du taux d'utilisation du processeur en réduisant notamment les attentes sur des entrées-sorties.
- Deux préoccupations:
 - ⌚ Comment permettre efficacement la cohabitation de plusieurs processus ?
 - ⌚ Comment assurer leur protection ?
- Solution :
 - ⌚ **Partition multiples**
 - ⌚ **Mémoire virtuelle**

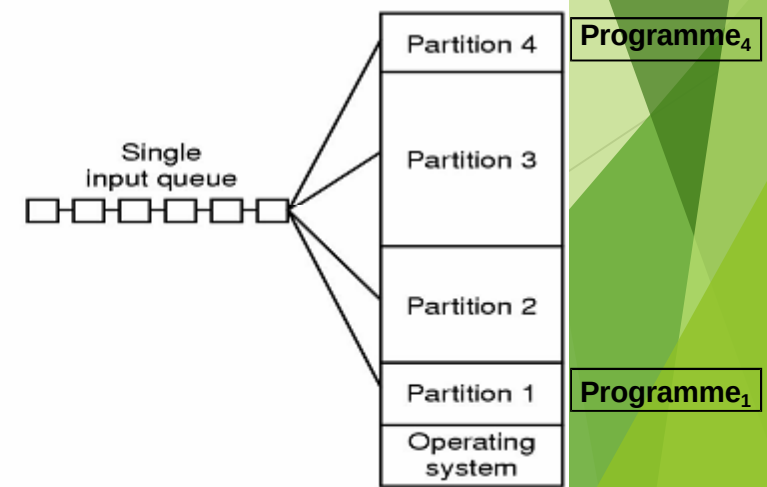
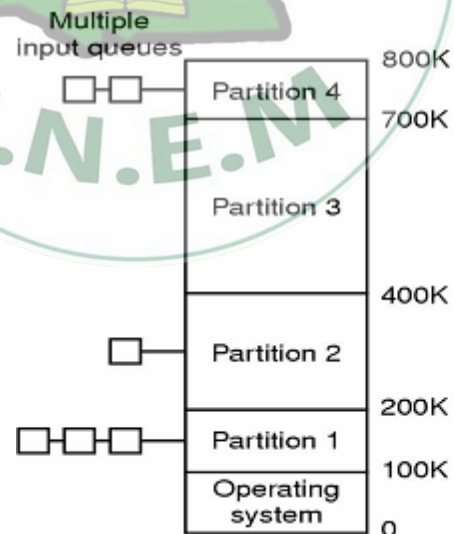


Stratégies d'allocation de la mémoire



Systeme multiprogrammé (Partition multiples) Partitions multiples statiques (fixes)

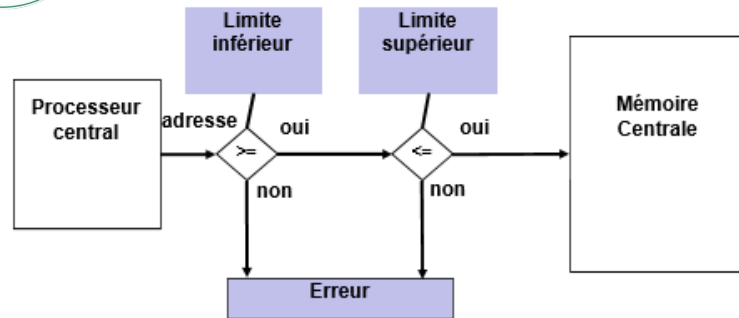
- ▶ La mémoire réservée aux utilisateurs est divisée en un ensemble de partitions.
- ▶ Le nombre et la taille des partitions sont fixés à l'avance (lors du chargement du système).
- ▶ Les partitions peuvent être égales ou non.
- ▶ Le choix du nombre et de la taille des partitions dépend du niveau de multiprogrammation voulu.
- ▶ On peut avoir une file par partition ou une file globale.
- ▶ L'unité d'allocation est une partition.



Stratégies d'allocation de la mémoire



Partitions multiples statiques (**Protection**)

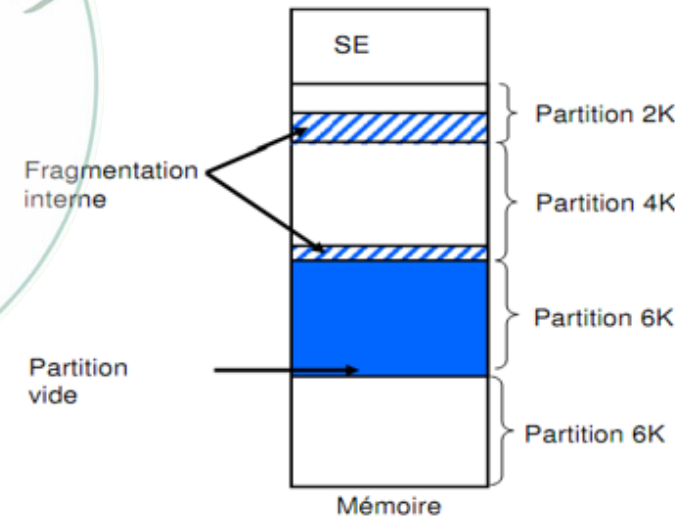


2^{ème} Solution: **Clé de Protection**

Partitions multiples statiques

Problème: **Fragmentation interne**

▢ Partie d'une partition non utilisée par un processus



Stratégies d'allocation de la mémoire

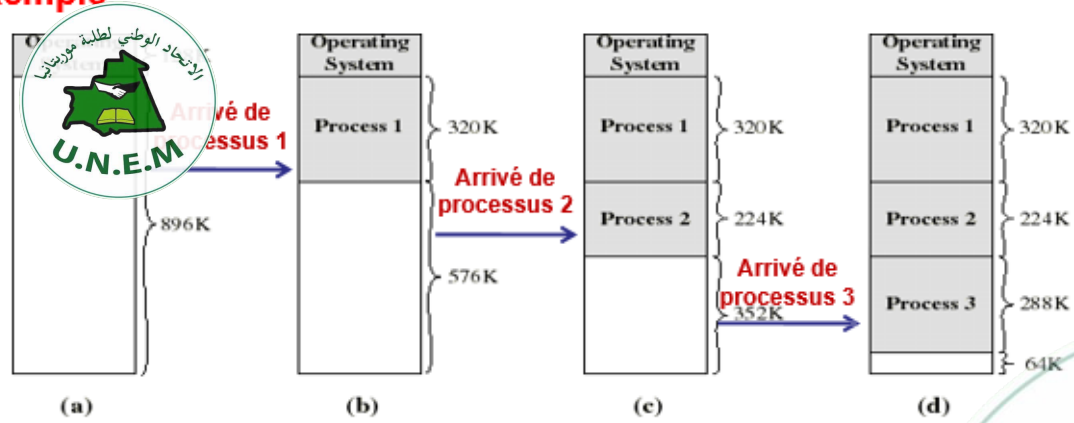


Système multiprogrammé (Partition multiples) Partitions multiples variables (dynamique)

- ▶ Initialement, l'espace mémoire réservée aux utilisateurs constitue une seule partition.
- ▶ Quand un nouveau processus doit être chargé, on lui alloue une zone contiguë de taille suffisamment grande pour le contenir.
- ▶ Le nombre et les tailles des partitions varient au cours du temps.
- ▶ Cette forme d'allocation conduit éventuellement à l'apparition de trous trop petits pour les allouer à d'autres processus : c'est la **fragmentation externe**
- **Solution:** Compactage (coûteux et exige que les processus soient relocalisables)

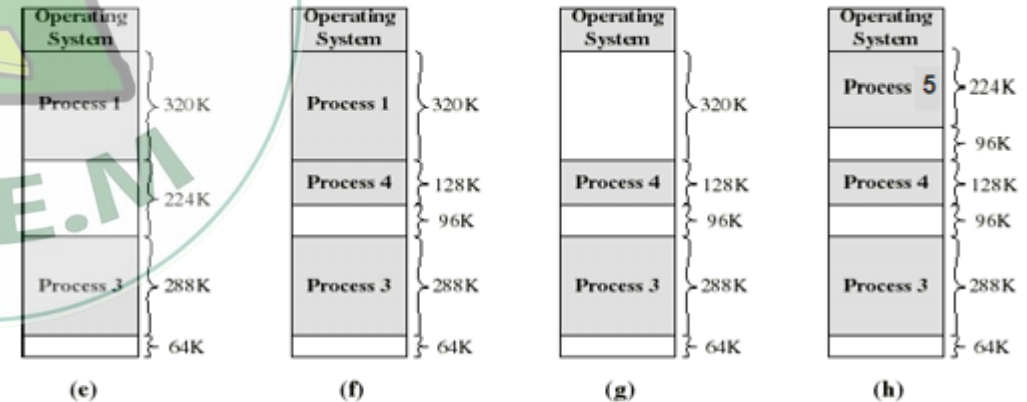
Partitions multiples variables (*dynamique*)

Exemple



- (d) Il y a un trou de **64K** après avoir chargé 3 processus: pas assez d'espace pour autre processus
- Si P2 **termine** alors le processus 4 de taille **128K** peut être chargé.

Partitions multiples variables (*dynamique*)



- (e-f) Processus 2 se termine, Processus 4 est chargé. Un trou de **224-128=96K** est créé (**fragmentation externe**)
- (g-h) P1 se termine, P5 (**224K**) est chargé à sa place: produisant un autre trou de **320-224=96K**... → **3 trous petits inutiles. 96+96+64=256K**
- **Compactage** : pour en faire un seul trou de 256K



Algorithme d'allocation de mémoire

- Il existe plusieurs algorithmes afin de déterminer l'emplacement d'un programme en mémoire (allocation contiguë). Le but de tous ces algorithmes est de maximiser l'espace mémoire occupé.
- **First Fit:** Le programme est mis dans le premier bloc de mémoire suffisamment grand à partir du début de la mémoire.
- **Next Fit:** Le programme est mis dans le premier bloc de mémoire suffisamment grand à partir du dernier bloc alloué.
- **Best Fit:** Le programme est mis dans le bloc de mémoire le plus petit dont la taille est suffisamment grande pour l'espace requis.
- **Worse Fit:** Le programme est mis dans le bloc de mémoire le plus grand.



- ▶ Soit Une mémoire Centrale composée de 5 partitions P1,P2,P3,P4,P5, ces partitions ont pour taille respectives 100, 500,200,300 et 600 K. Soit 4 processus A, B,C,D de tailles respectives 212, 417, 112 et 426 K.
- ▶ Donner les différents états de la mémoire centrale (sous forme de schémas) pour charger les processus A, B, C, et D en utilisant les algorithmes d'allocation suivants :
- ▶ A- FIRST-FIT, B- BEST-FIT, C- WORST-FIT,
- ▶ D- NEXT-FIT (on suppose que la zone où s'est arrêté l'algorithme d'allocation lors de la précédente recherche est la première partition de la mémoire.



2,
17,
C:112,
D:426

FIRST FIT

100	
500	A
200	C
300	
600	B

BEST FIT

B
C
A
D

WORST FIT

B
C
A

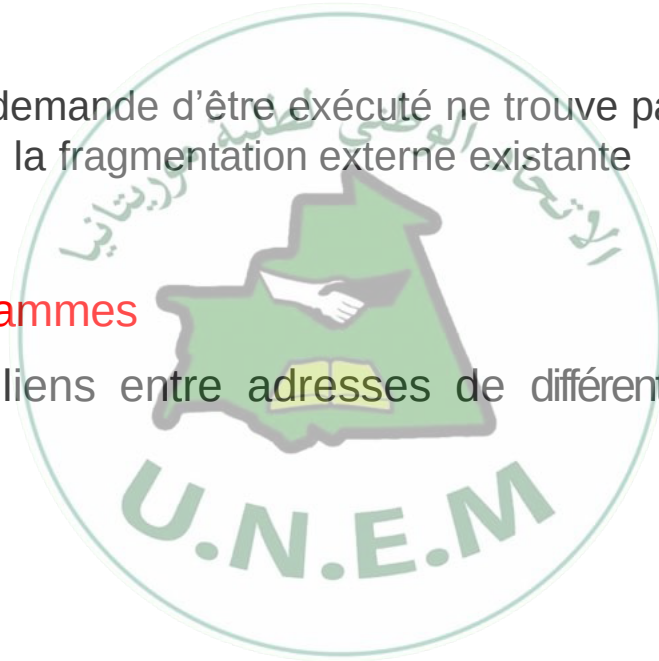
NEXT FIT

A
C
B



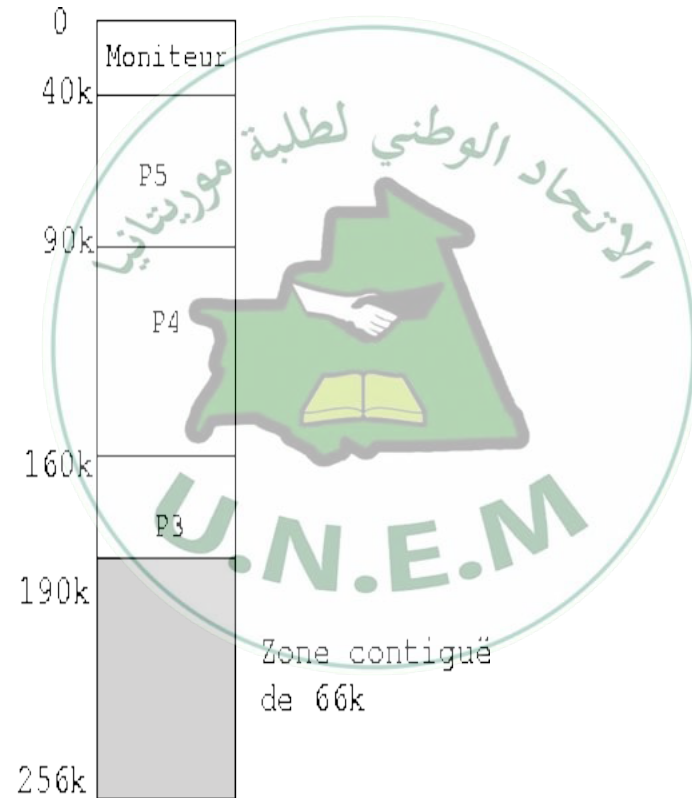
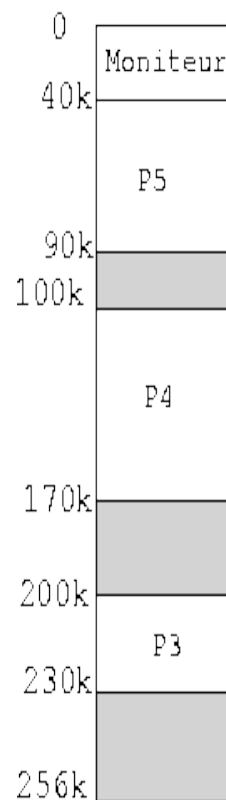
Compactage

- ▶ Une solution pour la fragmentation externe.
- ▶ Les programmes sont déplacés en mémoire de façon à combiner en un seul plusieurs petits trous disponibles
- ▶ Effectuée quand un programme qui demande d'être exécuté ne trouve pas une partition assez grande, mais sa taille est plus petite que la fragmentation externe existante
- ▶ Désavantages:
 - 🕒 **temps de transfert des programmes**
 - 🕒 besoin de rétablir tous les liens entre **adresses** de différents programmes





compactage On cherche à améliorer ces mécanismes en défragmentant la mémoire c'est à dire en déplaçant les processus en mémoire de façon à rendre contiguës les zones de mémoire libre de façon à pouvoir les utiliser.



Système de gestion de fichiers



- **Introduction** Le système de gestion de fichiers (SGF) est la partie la plus visible d'un système d'exploitation qui se charge de gérer le stockage et la manipulation de fichiers (sur une unité de stockage : partition, disque, CD, disquette).
- Un SGF a pour principal rôle de gérer les fichiers et d'offrir les primitives pour manipuler ces fichiers.



Partitionnement et Formatage



Partitionnement : consiste à « cloisonner » le disque. Il permet la cohabitation de plusieurs systèmes d'exploitation sur le même disque (il permet d'isoler certaines parties du système). L'information sur le partitionnement d'un disque est stockée dans son premier secteur (secteur zéro), le MBR (Master Boot Record).

Deux types de partitionnement :

- Primaire : On peut créer jusqu'à 4 partitions primaires sur un même disque.
- Etendue est un moyen de diviser une partition primaire en sous-partitions (une ou plusieurs **partitions logiques** qui se comportent comme les partitions primaires, mais sont créées différemment (pas de secteurs de démarrage))

Dans un même disque, on peut avoir un ensemble de partitions (multi-partition), contenant chacune un système de fichier (par exemple DOS et UNIX)

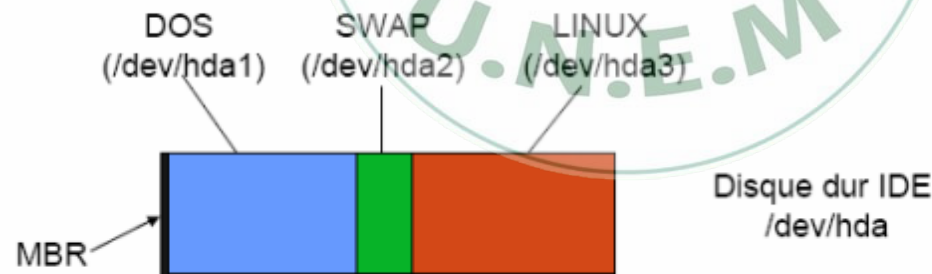


Figure 3. 1. Multipartition d'un disque



Formatage : Avant qu'un système de fichiers puisse créer et gérer des fichiers sur une unité de stockage, son unité doit être **formatée** selon les spécificités du système de fichiers. Le formatage inspecte les secteurs, efface les données et crée le répertoire racine du système de fichiers. Il crée également un **superbloc** pour stocker les informations nécessaires à assurer l'intégrité du système de fichiers.

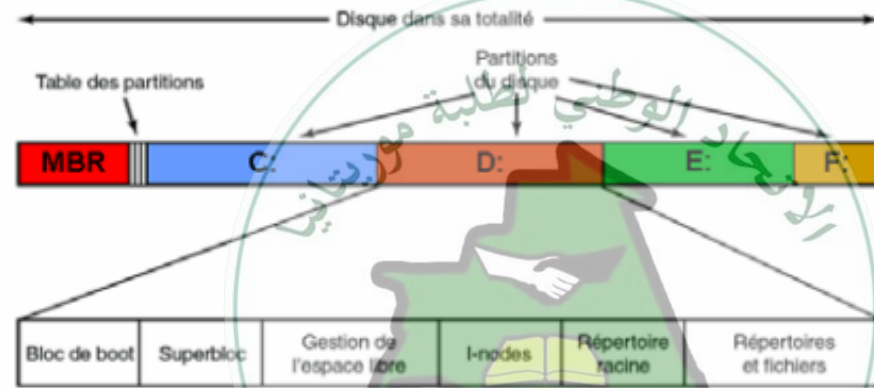


Figure 3. 2. Organisation du système de fichier

Un superbloc contient notamment : L'identifiant du système de fichiers (C:, D : ..), Le nombre de blocs dans le système de fichiers, La liste des blocs libres, l'emplacement du répertoire racine, la date et l'heure de la dernière modification du système de fichiers, une information indiquant s'il faut tester l'intégrité du système de fichiers.



concept de fichier

Un fichier est l'unité de stockage logique mise à la disposition des utilisateurs pour l'enregistrement de leurs données : c'est l'unité d'allocation. Le SE établit la correspondance entre le fichier et le système binaire utilisé lors du stockage de manière transparente pour les utilisateurs. Dans un fichier on peut écrire du texte, des images, des calculs, des programmes.

Les fichiers sont généralement créés par les utilisateurs. Toutefois certains fichiers sont générés par les systèmes ou certains outils tels que les compilateurs.

Afin de différencier les fichiers entre eux, chaque fichier a un ensemble d'attributs qui le décrivent. Parmi ceux-ci on retrouve : le nom, l'extension, la date et l'heure de sa création ou de sa dernière modification, la taille, la protection. Certains de ces attributs sont indiqués par l'utilisateur, d'autres sont complétés par le système d'exploitation.

Structure d'un répertoire



Un répertoire est une entité créée pour l'organisation des fichiers. En effet on peut enregistrer des milliers, des millions de fichiers sur un disque dur et il devient alors impossible de s'y retrouver. Avec la multitude de fichiers créés, le système d'exploitation a besoin d'une organisation afin de structurer ces fichiers et de pouvoir y accéder rapidement. Cette organisation est réalisée au moyen de **répertoires** également appelés **catalogues** ou **directory**.

Un répertoire est lui-même un fichier puisqu'il est stocké sur le disque et est destiné à contenir des fichiers. Du point de vue SGF, un répertoire est un fichier qui dispose d'une structure logique : il est considéré comme un tableau qui contient une entrée par fichier. L'entrée du répertoire permet d'associer au nom du fichier (nom externe au SGF) les informations stockées en interne par le SGF. Chaque entrée peut contenir des informations sur le fichier (attributs du fichier) ou faire référence à (pointer sur) des structures qui contiennent ces informations.

Exemple 3. 1

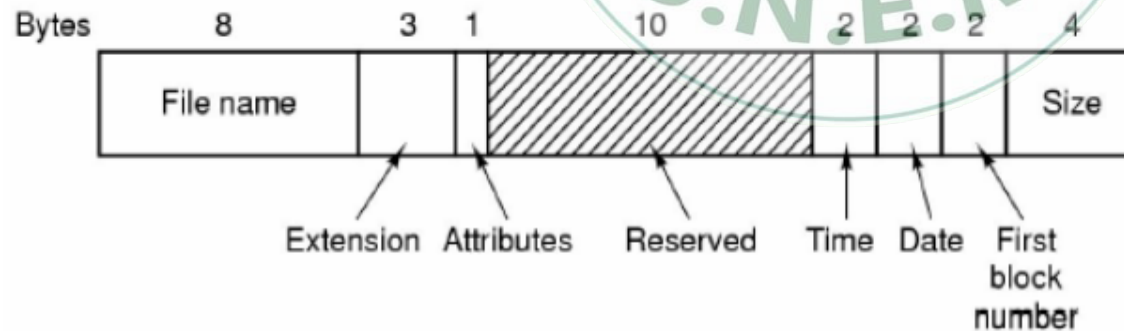


Figure 3. 3. Structure d'un répertoire : cas de MS-DOS (32 octets)

Commande MS-DOS



Introduction

DOS est le système d'exploitation le plus connu, sa version la plus commercialisée est celle de Microsoft, baptisée MS-DOS (Microsoft Disk Operating Système).

MS-DOS a vu le jour en 1981 lors de son utilisation sur un IBM PC. MS-DOS est un système d'exploitation mono-tâche et mono-utilisateur.

II. Définitions

➤ Commande

Une commande est une suite de caractères qui sont compréhensibles par l'ordinateur (exemple : Dir, Del, Copy, MD, Exit, Time, Date, Cls, etc).

➤ Fichier

Un fichier est une suite d'octets enregistrés sur un périphérique de masse (disque dur, disquette, clé USB, CR-ROM, ...). Le nom d'un fichier est composé de trois parties : nom, point et l'extension.

Exemple de fichiers :

cv.doc, photo.gif, classeur1.xls.

L'extension détermine le type de fichier par exemple :

TXT désigne les fichiers textes ;

BAT désigne les fichiers batchs ;

EXE désigne les fichiers exécutables.

Commande MS-DOS



➤ Répertoire

Les répertoires sont des regroupements de fichiers et de sous répertoires. Cela permet à l'utilisateur de classer ses fichiers comme il le ferait avec des feuilles dans un classeur. Ainsi, nous pouvons mettre tous les fichiers relatifs à un sujet dans un même dossier (répertoire).

➤ Chemin d'accès

Pour accéder à un fichier sur un disque, il ne suffit pas de connaître juste son nom, il faut aussi connaître sa localisation dans l'arborescence. Ainsi, "C:\ENCG\IMAGES\LOGO.GIF" désigne le fichier "LOGO.GIF" dans le répertoire "IMAGES" du répertoire "ENCG" du disque "C:".

➤ Invite de commandes MS-DOS

L'invite de commandes est l'environnement qui interprète les commandes saisies par l'utilisateur.

Pour lancer l'invite de commandes (figure suivante), cliquer sur le menu **Démarrer/ Programmes/ Accessoires/ Invite de commandes**.

```
C:\>ver  
Microsoft Windows [version 10.0.22621.4317]  
C:\>|
```


Commande MS-DOS



de commandes ou Prompt indique l'unité et le répertoire en cours. Par exemple : `C:\WINDOWS>` signifie que vous êtes sur l'unité logique C et sous le répertoire WINDOWS.

Note :

Vous pouvez accéder à l'invite de commandes en procédant comme suit :

- Sélectionnez **Démarrer** puis **Exécuter**.
- Tapez **CMD** ou dans certaines versions de Windows **COMMAND**.
- Cliquez sur **OK**.

III. Gestion des répertoires sous DOS

➤ Commande MD

La commande MD (Make Directory) ou MKDIR permet de créer un répertoire.

Syntaxe :

MD nom_repertoire

MKDIR nom_repertoire

Exemple :

On suppose que vous êtes en `C:\>`. Créer un répertoire ENCG dans le disque local C.

C:\>md encg

Remarques :



• C'est affiché par l'ordinateur et la commande `md encg` est saisie par l'utilisateur. Dans la suite, on va souligner ce qu'est affiché par l'ordinateur

pour le différencier de la commande saisie par l'utilisateur.

☐ Vous pouvez créer le répertoire `encg` en utilisant l'explorateur Windows (l'interface visuel de Windows).

☐ Commande `DIR`

La commande `DIR` affiche la liste des fichiers et des sous répertoires d'un répertoire (dossier).

Syntaxe :

`Dir [nom_dossier] [commutateurs]`

Ce qu'est entre crochets `[]` est optionnel.

Exemple de commutateur :

`/P` : affichage page par page

`/W` : affichage sur 5 colonnes



exemple :

C:\> dir : affiche le contenu du disque local C

C:\> dir *.txt : affiche tous les fichiers dont l'extension est txt et qui sont dans le disque local C.

C:\> dir windows : affiche le contenu du dossier Windows.

C:\WINDOWS> dir : affiche le contenu du dossier Windows.

Remarques :

- ▣ Chaque fois qu'on tape une commande, on valide la commande par la touche ENTREE du clavier.
- ▣ Sous MS-DOS, il n'y a pas de différences entre une commande saisie en majuscule et une commande saisie en minuscule. Par contre, sous Unix ou Linux, il y a une différence.



Commande CD

La commande CD ou ChDir (Change Directory) change de répertoire.

Syntaxe :

CD nom_repertoire

Exemple :

On suppose que vous êtes en C:\> et vous souhaitez entrer au dossier Windows.

C:\> cd Windows

L'ordinateur affiche C:\WINDOWS>.

Commande CD \

La commande CD \ permet un accès rapide à la racine.



Exemple :

Suppose que vous êtes en C:\WINDOWS\SYSTEM32> et vous souhaitez revenir à C:\>.

C:\WINDOWS\SYSTEM32> cd \

L'ordinateur affiche C:\>.

Note :

/ : est appelée Slash.

\ : est appelée Anti-slash.

Pour afficher à l'écran le caractère Anti-slash « \ », maintenir la touche ALTGR

enfoncé et cliquez sur la touche 8 du clavier.

□ Commande CD ..

La commande CD .. permet de remonter d'un niveau (se placer dans le répertoire parent).

Exemple :

C:\WINDOWS\SYSTEM32> cd ..

L'ordinateur affiche C:\windows>.

C:\WINDOWS> cd ..

L'ordinateur affiche C:\>.





Remarque 1 :

Série des commandes suivantes :

```
C:\> cd dossier1
```

```
C:\ dossier1> cd dossier2
```

```
C:\ dossier1\ dossier2> cd dossier3
```

```
C:\ dossier1\ dossier2\ dossier3> cd dossier4
```

```
C:\ dossier1\ dossier2\ dossier3\ dossier4>
```

est équivalente à :

```
C:\> cd dossier1\ dossier2\ dossier3
```

qu'est équivalente à :

```
C:\> cd C:\ dossier1\ dossier2\ dossier3\ dossier4
```

On suppose que Dossier1, dossier2, dossier3 et dossier4 ceux sont des répertoires

(dossiers) qui existent dans votre machine.

Remarque 2 :

```
C:\> MD Dossier1
```

Cette commande crée le Dossier 1 et l'ordinateur affiche C:\> et non pas C:\Dossier1>..



Si le nom d'un dossier contient des espaces, on met le nom du dossier entre deux tirets. Par exemple :

```
C:\> cd "Documents and Settings"
```

L'ordinateur affiche C:\Documents and Setting>.

☐ Changer l'unité physique ou logique

Pour changer l'unité physique ou logique, il suffit de taper la lettre du lecteur suivi de ":".

Exemple :

Entrer à la disquette. On suppose que vous êtes en C:\>.

```
C:\> a :
```

L'ordinateur affiche A:\>.

Exemple :

Revenir au disque local C.

```
A:\> c :
```

L'ordinateur affiche C:\>.

Exemple :

Donnez la commande pour entrer au CD-ROM. On suppose que le CD-ROM est désigné par la lettre E et que vous êtes en C:\>.

```
C:\> E :
```

L'ordinateur affiche E:\>.





Exercice :

Donner la liste des commandes permettant d'afficher le contenu de la disquette ?

Vous êtes en C:\>.

C:\> a :

A:\> dir

Note :

En pratique la touche F3 permet de réafficher la dernière commande. Les touches de direction aussi.

☐ **Commande RD**

La commande RD (Remove Directory) permet de supprimer un répertoire vide.

Syntaxe :

RD nom_repertoire

Exercice :

Créer le répertoire Etudiant dans le disque local C et supprimer le. On suppose que vous êtes en C:\>.

C:\> md Etudiant

C:\> rd Etudiant



Remarque : Pour supprimer un dossier vous devez sortir de ce dossier.

Commande DELTREE

La commande DELTREE supprime un dossier et son contenu. Syntaxe :

DELTREE nom_repertoire **Remarque : Cette commande n'est reconnue par tous les ordinateurs car il ya plusieurs versions de MS-DOS..**





